

Abstractizare

`abstract` · `interface` · `static` — peste ierarhiile tale de figuri și de orar · Java · Adrian

Unde lucrezi: repo nou `java-abstractizare`. Copiază acolo, așa cum sunt, pachetele tale din proiectul de moștenire: figurile geometrice în `app.figuri` și intervalele de orar în `app.orar`. Bug-urile semnalate în runda 2 de review se rezolvă **în repo-ul vechi** — aici ne interesează altceva.

Nou, ai voie: `abstract` pe clasă și pe metodă, `interface + implements`, `static`, `static final`, constructor `private`.

Încă NU folosim: metode `default` sau `static` în interfețe, generice proprii (`<T>`), `enum`, `record`, `clone()`, `equals` / `hashCode`, `fișiere`, `try/catch`, `stream-uri`.

Regula fișei: fiecare exercițiu are un rând **Gata când**. Se verifică prin **rulare** și prin ce scrie compilatorul, nu prin citirea codului. Mesajele de eroare se notează pe fișă, cuvânt cu cuvânt — sunt jumătate din exercițiu.

Partea 1 — `abstract` : obiectul care n-ar trebui să existe

1. Ce se întâmplă când instanțiezi superclasa

nivel 1

În `App`, rulează exact asta:

```
Desen d = new Desen(new ArrayList<>(List.of(
    new Cerc(new Punct(2, 3), 5),
    new Figura()
)));
d.afisare();

Orar orar = new Orar();
orar.adaugaInterval(new OraCurs(8, 50, "Mate", "A12"));
orar.adaugaInterval(new Interval(9, 10));
orar.afisare();
```

Notează pe fișă **exact** ce apare pe rândul lui `new Figura()` și pe rândul lui `new Interval(9, 10)`.

Apoi scrie `abstract` în fața lui `class Figura` și a lui `class Interval` și încearcă să compilezi aceleași linii. Notează mesajul și numărul liniei.

Gata când: ai amândouă notate una sub alta — ieșirea de dinainte și eroarea de compilare de după.

Înainte de `abstract`, programul rula fără să se plângă. Ce anume „mergea” acolo — și cui îi era de folos rândul acela?

2. Metoda pe care o poți uita

nivel 1

Adaugă în `app.orar` un tip nou de interval: **Examen** — ora de început, durata în minute, materia, sala și nota maximă. Scrie-l pe tiparul lui `OraCurs`, dar **nu scrie în el** `getTipInterval()`. Pune un examen în orar și rulează.

Notează rândul afișat pentru examen.

Apoi, în `Interval`, transformă `getTipInterval()` în metodă abstractă — declarată, fără corp — și compilează din nou.

Gata când: ai notat rândul din prima variantă și eroarea de compilare din a doua, împreună cu numele clasei care apare în ea.

În prima variantă nimeni nu s-a plâns. Cine ar fi trebuit să observe că orarul are un interval fără tip — tu, compilatorul, sau cel care citește orarul afișat?

3. Ce rămâne scris într-o clasă abstractă

mai greu

Acum `afisare()` există în `Interval` și e rescrisă în `OraCurs`, `Pauza`, `ActivitateLibera` și `Examen` — fiecare cheamă `super.afisare()` și mai adaugă ceva.

Rescrie `Interval` astfel încât **formatul rândului să existe într-un singur loc**, iar subclasele să nu mai conțină deloc metoda `afisare`. Tot ce diferă între tipuri rămâne declarat abstract în `Interval`.

Gata când: `grep -c afisare` pe fiecare subclasă dă `0`, iar ieșirea programului e identică cu cea de dinainte — compar-o rând cu rând, nu din memorie.

O clasă abstractă are voie și cu metode gata scrise, și cu metode doar declarate. Din rândul afișat, ce bucată e aceeași pentru toate intervalele și ce bucată o știe numai subclasa?

Partea 2 — `interface` : contractul peste ierarhii diferite

4. Aceeași capacitate în trei locuri fără rudă comună

mai greu

În codul tău, „știu să mă copiez” apare de trei ori, sub două nume: `Figura.duplicare()`, `Interval.duplicate()` și `OraCurs.duplicate()`. `Figura` și `Interval` n-au nicio superclasă comună, iar `OraCurs` nu e nici figură, nici interval.

Scrie o interfață cu o singură metodă pentru capacitatea asta și fă toate trei s-o implementeze, **sub un singur nume de metodă**. Apoi scrie, într-o clasă din afara celor două pachete, o metodă care primește o listă de astfel de obiecte și întoarce lista copiilor.

În `App`: pune în **aceeași** listă un `Cerc`, o `OraCurs` și un `OraCurs`, cheam-o și afișează originalele și copiile.

Regula exercițiului: în metoda care copiază n-ai voie nici `instanceof`, nici `cast`, nici `if` după tip.

Gata când: lista din `App` e declarată o singură dată, ține obiecte din ambele pachete, și programul afișează 3 originale plus 3 copii.

Și `extends` ți-ar da un tip comun. Încearcă în minte: ce superclasă ai putea pune deasupra lui `Cerc` și a lui `OraCurs` deodată — și ce ar moșteni un `OraCurs` de la ea?

5. Un părinte, câte contracte vrei

mai greu

a) Scrie în `Eticheta` linia:

```
public class Eticheta extends Dreptunghi, Figura {
```

Compilează, notează mesajul exact, apoi revino.

b) Acum `Eticheta` moștenește `Dreptunghi` și în plus ține un câmp `Dreptunghi` — aceleași două puncte sunt păstrate în două locuri. Scrie o a doua variantă de `Eticheta`, care **nu** moștenește `Dreptunghi`, ci doar ține unul pe dinăuntru.

Ca varianta b) să încapă tot într-un `Desen`, `Desen` nu mai poate cere o clasă anume pentru elementele lui. Decide tu ce cere în loc.

Gata când: ambele variante stau într-un `Desen` și se afișează, iar pentru varianta b) `translate` mută eticheta — dovedește-o afișând coordonatele înainte și după.

În varianta a), câte perechi de puncte are de fapt un obiect `Eticheta`? Numără câmpurile, nu metodele. Și dacă cineva cheamă un setter, care pereche se schimbă?

6. Interfața ca întrebare „ce poți face”, nu „ce ești”

mai greu

Punct și Linie n-au arie. Cerc și Dreptunghi au. Scrie o interfață pentru „figură care are arie” și fă doar cele două s-o implementeze.

În `Desen`, scrie `arieTotala()`, care adună ariile figurilor care au arie și le trece cu vederea pe celelalte.

Regula exercițiului: în `arieTotala()` ai voie `instanceof` cu numele **interfeței**, dar nu cu numele unei clase (`Cerc`, `Dreptunghi`).

Gata când: adaugi un `Triunghi` cu arie, îl pui în desen, și `arieTotala()` îl numără **fără nicio modificare** în `Desen`.

Pune una lângă alta cele două verificări: `instanceof Cerc` și `instanceof` -ul cu interfața. Una întreabă „ce ești”, alta „ce poți face”. Care dintre ele se strică atunci când apare o clasă nouă?

Partea 3 — `static` : ce ține de clasă, nu de obiect

7. Numărul 60 și numărul 1440

nivel 1

În `Interval`, `60` și `1440` sunt scrise de mână în mai multe locuri. Adună-le într-un singur loc, ca valori care aparțin **clasei**, nu obiectului, și pe care nimeni nu le mai poate schimba după ce au fost scrise o dată.

În `App`, afișează valoarea „minute pe zi” **fără să creezi niciun** `Interval`.

Gata când: `grep -n "60\|1440" Interval.java` nu mai întoarce nimic în afara liniilor de declarare, iar linia din `App` care afișează valoarea nu conține `new`.

Scrie undeva mai încolo, într-o metodă, o atribuire peste valoarea aia. Ce spune compilatorul — și de ce e bine că o spune?

8. Constructorul care are un nume

mai greu

Scrie două metode care construiesc obiecte fără să fie constructori:

- în `Interval`, una care primește ora ca text (`"8:30"`) și durată în minute, și întoarce un interval;
- în `Punct`, una care întoarce punctul (0, 0).

Cheamă-le din `App` pe **numele clasei**, nu pe un obiect.

Apoi, în metoda din `Interval`, încearcă să folosești câmpul `oraInceput` direct, fără obiect. Notează mesajul exact al compilatorului.

Gata când: ambele metode merg chemate pe numele clasei, iar mesajul de eroare e notat pe fișă și explicat de tine într-o singură frază.

O metodă obișnuită primește pe ascuns un parametru în plus: obiectul pe care a fost chemată. Ce lipsește într-o metodă chemată pe numele clasei — și de ce tocmai de aia n-are ce să citească?

9. Contorul care trebuie să țină minte între obiecte

provocare

Dă-i fiecărui `Interval` un id unic, numerotat de la 1, primit automat la creare. Creează 3 intervale și afișează id-urile.

Apoi fă a doua variantă, în care contorul e câmp **obișnuit** al obiectului, și rulează la fel. Notează cele două serii de id-uri una sub alta.

La final: după `orar.duplicate()`, ce id-uri au intervalele din copie? Decide ce e corect pentru un orar duplicat — să păstreze id-urile sau să primească altele — și scrie într-un rând de ce.

Gata când: ai pe fișă cele două serii de id-uri și id-urile copiei, plus decizia motivată.

*Contorul trebuie să știe câte obiecte s-au creat **înainte** celui de acum. Un câmp al obiectului se naște odată cu obiectul — de unde ar afla el ce a fost înainte de el?*

10. Clasa pe care n-o instanțiezi niciodată

provocare

Ora afișată se construiește din minute în mai multe locuri din `Interval`. Scrie o clasă `Timp` care să conțină **doar** transformările între minute și text, și fă-o **imposibil de instanțiat din afară**. Cheam-o din `Interval` și, separat, direct din `App`.

Gata când: `grep -rn "new Timp"` pe tot proiectul dă 0; `Interval` nu mai face socoteli de ore și minute în afara apelurilor către `Timp`; iar `App` afișează un text de oră fără să creeze niciun `Interval`.

La final, răspunde în scris: de ce ai pune constructor privat pe o clasă pe care oricum n-aveai de gând s-o instanțiezi?

Fără constructor scris de tine, clasa are unul implicit și public. Ce-l oprește pe altcineva — sau pe tine, peste o lună — să scrie `new Timp()` și să aștepte ceva de la obiectul acela?